

Теория: Высокопроизводительные TCP- серверы на System.Net.Sockets

C# Developer. Advanced



Проверить, идет ли запись

Меня хорошо видно & слышно?





- ❖ Lead Software Engineer в S&P Global;
- ❖ Руководитель четырех .Net курсов в Отус;
- ❖ Автор книги [Functional Programming with C#](#);
- ❖ Автор трёх ютуб каналов: [1](#), [2](#), [3](#) и сообщества [Pro .NET](#) (для подключения пишите в личку);
- ❖ С 2000 года в программировании (последние 15 лет с C#);
- ❖ Первое высшее: программирование, а второе - управление организацией;
- ❖ Работал в маленьких стартапах, крупных международных компаниях, частных и бюджетных организациях, научно-исследовательском институте, ВУЗе;
- ❖ Из общеизвестных в России - два года в Лаборатории Касперского.

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе **OTUS C#-in-de**



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу



Маршрут вебинара

Введение в System Design

Что такое кэш

Проектирование High-Level архитектуры

Разбор компонент

Выявление рисков

Вопросы и ответы



Вопросы на конец занятия

1. Что такое Socket на самом деле?

2. Чем плохи обёртки над сокетами?

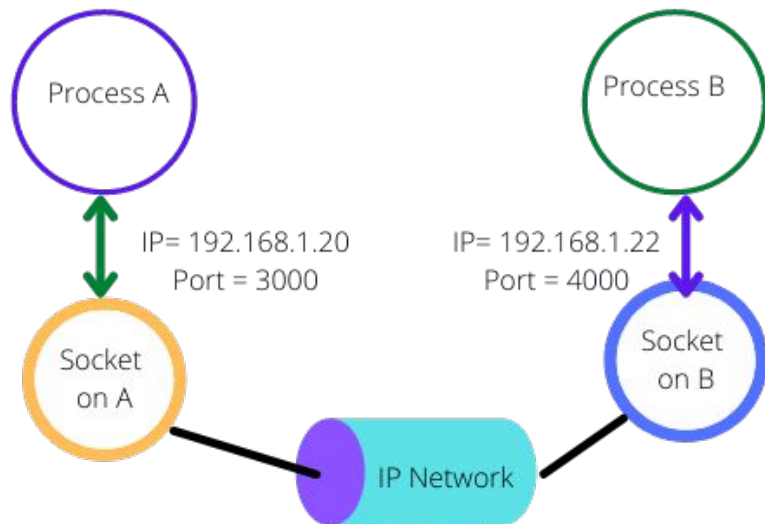
3. Как классический подход к разработке серверов приводит к краху под нагрузкой?

System.Net.Sockets



Сокет

— это программный интерфейс (точка соединения), через который происходит обмен данными между процессами и программами. Состоит из IP-адреса и номера порта. Например, 192.168.1.100:8080 — это сокет, где 192.168.1.100 — IP-адрес компьютера, а 8080 — номер порта.



Зачем нужны сокеты?

Две основные задачи



Передача данных по сети

Браузер общается с веб-сервером или
мобильное приложение получает
данные с сервера

Зачем нужны сокеты?

Две основные задачи



Передача данных по сети

Браузер общается с веб-сервером или мобильное приложение получает данные с сервера

Связь между приложениями на одном компьютере

Многие служебные программы используют внутренние сокеты для передачи данных операционной системе

Типы сокетов

Потоковые сокеты

(stream sockets)

Обеспечивают надёжную доставку данных в виде непрерывного потока байтов, работают на протоколе TCP.

Может обслуживать миллионы соединений на одном порту одновременно, так как каждое TCP-соединение идентифицируется уникальной комбинацией четырёх параметров: IP-адрес источника + порт источника + IP-адрес назначения + порт назначения.



Типы сокетов

Потоковые сокеты (stream sockets)

Обеспечивают надёжную доставку данных в виде непрерывного потока байтов, работают на протоколе TCP.

Может обслуживать миллионы соединений на одном порту одновременно, так как каждое TCP-соединение идентифицируется уникальной комбинацией четырёх параметров: IP-адрес источника + порт источника + IP-адрес назначения + порт назначения.

Датаграммные сокеты (datagram sockets)

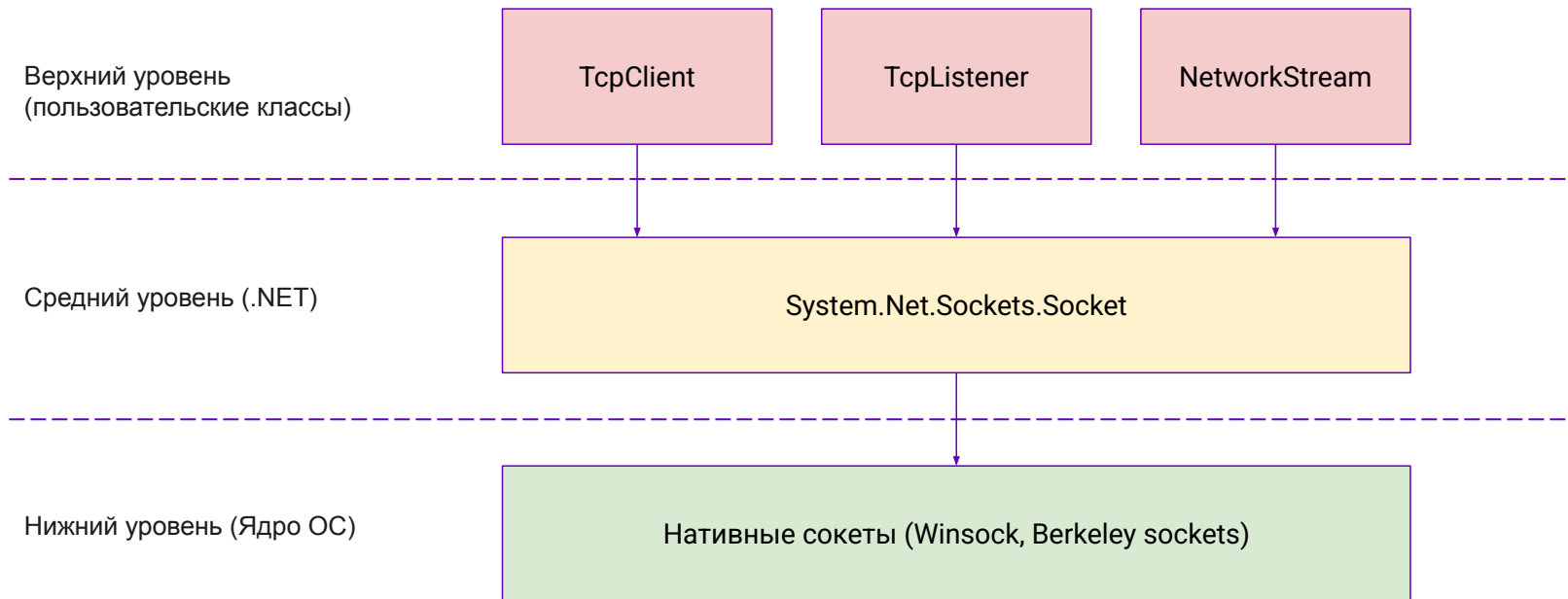
Обеспечивают быструю передачу данных без гарантии доставки, работают на протоколе UDP.

По умолчанию только один UDP-сокет может быть привязан к порту. Теоретически можно привязать несколько сокетов к одному порту, но принимать данные будет только последний привязанный сокет.

Важно: Сокеты являются двунаправленными (full duplex), то есть данные могут передаваться в обоих направлениях одновременно.



Уровни абстракции: Простота vs. Контроль



Демо

Алгоритм Нейгла

— алгоритм для снижения количества пакетов, отправляемых по сети.

```
если есть новые данные для отправки
  если размер окна  $\geq$  MSS и доступные данные  $\geq$  MSS
    отправить полный MSS сегмент сейчас
  иначе
    если есть неподтвержденные данные в канале
      поместить данные в буфер до получения подтверждения
    иначе
      отправить данные немедленно
  конец если
конец если
конец если
```

*MSS — максимальный размер блока данных для TCP-пакета.

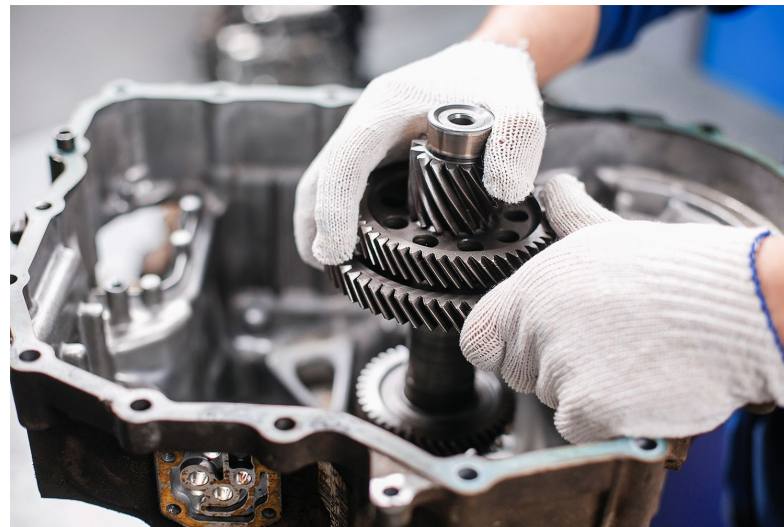
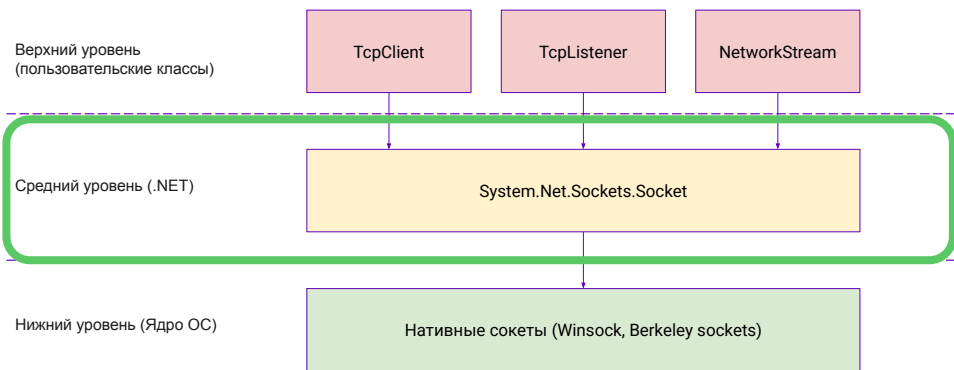
```
socket = new Socket(AddressFamily.InterNetwork, SocketType.Stream, ProtocolType.Tcp);
socket.NoDelay = true;
```



Демо

SocketAsyncEventArgs

- класс для высокопроизводительной работы с сокетами.



Демо

Какие решили проблемы?

- Обработка запросов по очереди
- Слишком много потоков
- Нагрузка на память (на каждый поток ~1Mb стека, 10,000 потоков == 10 Gb памяти)
- Нагрузка на процессор
- Исчерпание пула потоков (зависание сервера)
- Загрузка GC ($10,000 \text{ клиентов} * 10 \text{ сообщ/сек} * 1000 \text{ байт/сообщ} \approx 100 \text{ МБ/сек}$)
- Отсутствие детального контроля (?)



Выводы

Сводная таблица подходов

Подход	Пропускная способность	GC-давление	Сложность	Рекомендуемое применение
<code>NetworkStream + async/await</code>	Средняя	Высокое	Низкая	Простые клиенты, серверы с <1000 <i>неактивных</i> соединений, прототипы.
<code>Socket + ValueTask</code>	Выше среднего	Умеренное	Средняя	Оптимизированный <code>async/await</code> для сценариев, где <code>Pipelines</code> избыточны.
<code>SocketAsyncEventArgs (SAEA)</code>	Максимальная	Нулевое	Экстремальная	Разработка фреймворков (Kestrel), ultra-low latency.
<code>System.IO.Pipelines</code>	Почти макс.	Очень низкое	Выше среднего	Простой в использовании и быстрый способ разработки высоконагруженных TCP-серверов.



Вопросы для проверки знаний

1. Что такое Socket на самом деле?

2. Чем плохи обёртки над сокетами?

3. Как классический подход к разработке серверов приводит к краху под нагрузкой?



**Отправьте в чат эмодзи, который
отражает ваше настроение
после занятия**



**Продолжите высказывание: теперь
я умею/знаю...**



**Что планируете использовать
в работе?**

Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️

Вопросы



если есть вопросы



если вопросов нет

OTUS

